Ders 3 Çalışma Özeti

Ders 3 Çalışma Özeti

Genel Konular

- Prototype: amaç ve örnek problem (ekrandan, 00:10:00 ve 00:13:00)
  - Amaç: oluşturulacak nesnelerin türlerini belirlemek için bir prototip nesne kullanmak ve yeni nesneleri bu prototipi kopyalayarak oluşturmak. Hoca, bu mantıkla çalışan programlama dilleri de olduğunu, kalıbın onların çözdüğü sorunları çözdüğünü söyledi.
  - GoF örneği: grafik uygulamalarına yönelik hazır bir çerçeve programı (framework) kullanılarak müzik besteleme programı yazılacak. Çerçevede soyut `Graphic` sınıfı, kontrol düğmelerinin yer aldığı paletleri simgeleyen soyut `Tool` sınıfı ve bundan türetilmiş, çizim alanına yeni grafik şekilleri ekleyen `GraphicTool` sınıfı vardır.
  - Sorun: çerçeve programının nota sınıfları hakkında önceden bilgisi olamaz. `GraphicTool` bilmediği sınıflardan çizim nesnelerini nasıl oluşturur?
  - Çözüm: `GraphicTool`'un bir `prototype` üyesi (`Graphic` türünden) vardır. `Manipulate()` içinde `p = prototype->Clone()` ile kopya alınır, kullanıcı fareyi sürükledikçe `p->Draw(new position)` çağrılır, sonunda `p` çizime eklenir. `Graphic` altında `Staff` (porte) ve soyut `MusicalNote`, onun altında `WholeNote`, `HalfNote` vardır; her biri `Clone()` içinde kendisinin kopyasını döndürür.
  - Hoca, şemadaki ilişkinin bire çok sahiplik (parça-bütün) gibi düşünülebileceğini; şema üzerinde kırmızı çizginin üstünün çerçeveye, altının müzik uygulamasına ait olduğunu gösterdi.
- Aynı problemin başka kalıplarla çözümü (hocanın tartışması)
  - Abstract Factory: aileler nota türlerine (tam, yarım, ters, düz; 4 tür), üyeler notalara (7 nota) göre kurulabilir. Hoca, sayısı az olanı aile yapmanın (4 ailenin 7 üyesi) daha mantıklı olduğunu söyledi.
  - Factory Method da benzer kurulabilir. İkisinin de sorunu, her nota ya da nota türü için ayrı fabrika sınıfı gerektirmesidir (7 ya da 4 fabrika). Hoca bunları daha az kullanışlı buldu.
- Prototype kalıp yapısı (ekrandan, 00:17:30)
  - `Client` (`Operation()` içinde `p = prototype->Clone()`), soyut `Prototype` (soyut `Clone()`), `ConcretePrototype1` ve `ConcretePrototype2` (`Clone()`: kendisinin kopyasını döndürür).
  - İstemci bir işlem yapmadan önce prototipin kopyasını alır, işi kopya üzerinden yapar. Tek bir somut prototip de yeterli olabilir; farklı davranışlar gerekiyorsa kardeş sınıflar eklenir.
- Prototype'ın uygulanabileceği durumlar (ekrandan, 00:20:00)
  - Bir sistemin kendi ürünlerinin nasıl oluşturulduğundan, birleştirildiğinden ve sunulduğundan bağımsız olması istendiğinde ve ayrıca şu durumlardan biri söz konusuysa (Factory Method'dan farkı bu koşullar sağlar):
    - Oluşturulacak sınıflar çalışma anında belirlenecekse (ör. oyunda bölüm sonu canavarı, uçan düşman, büyücü düşman gibi türlerin oyun sırasında değişmesi).
    - Ürünlerin sınıf hiyerarşisine paralel bir factory hiyerarşisi kurmak istenmiyorsa.
    - Ürün sınıfının örneklerinin durum uzayı sınırlıysa (az sayıda prototip yeter).
  - Üç alt koşuldan birinin olması kalıbı seçmek için yeterlidir.
  - Kalıp, C++ gibi sınıfların doğrudan işlenebilen varlıklar olmadığı dillerde daha çok yarar sağlar. Smalltalk gibi geç ilişkilendirme (late binding) ya da Java gibi yansıtma (reflection) yetenekleri olan dillerde kalıp yerine dilin yetenekleri kullanılabilir.

- Prototype tartışmaları (ekrandan, 00:27:50)
  - Sığ kopyalama (shallow copy): yalnız nesnenin kendisi kopyalanır. Derin kopyalama (deep copy): nesnenin ilişkide olduğu nesneler de kopyalanır.
  - Sığ kopyalama yetersiz kalabilir; derin kopyalamada çevrimsellik (ör. çalışan → amir → amirin altındakiler → çalışan) sonsuz döngüye yol açabilir.
  - Klonların üyelerine değer atanması için `Prototype` sınıfında ek metotlar tanımlamak gerekebilir. Kopya kurucu (copy constructor) yazmak da işe yarayabilir.
- Prototype örneği: oyun için level editörü (Head First Design Patterns'ten) (ekrandan, 00:30:00, 00:35:10, 00:38:30, 00:40:00)
  - Haritaya çok çeşitli canavarlar kolayca eklenmek isteniyor. Canavar oluşturmak zordur, çok durum bilgisi vardır: `HP`, `DEX`, `AGL`, `LUCK`, `CHR`.
  - Soyut `Monster` (Prototype): protected `int` statlar ve `String type`, `+ clone(): Monster {abstract}`, `+ getType()`, protected kurucu `# Monster(int, int, int, int, int)`, public getter'lar ve protected setter'lar.
  - `GenericMonster` (ConcretePrototype): kurucu `super(...)` çağırır ve `type = "Generic Monster"` atar; `clone()` aynı statlarla yeni bir `GenericMonster` döndürür.
  - `MonsterRegistry`: `HashMap<String, Monster> monsters`; `findMonster(type)` → `monsters.get(type)`, `registerMonster(m)` → `monsters.put(m.getType(), m)`.
  - `MainApp` (Client): registry'ye `GenericMonster(10, 10, 10, 10, 10)` kaydeder, `findMonster("Generic Monster").clone()` ile kopya alır, `setHP(100)` ve `setType("Bolum sonu canavarı")` yapıp yeniden kaydeder.
- Singleton (ekrandan, 00:46:10 ve 00:50:00)
  - Amaç: bir sınıfın yalnız bir örneğinin bulunmasını sağlamak ve bu nesneye ortak bir erişim noktası vermek.
  - Örnekler: bir bilgisayarda birden fazla yazıcı tanımlı olabilir ama tek bir yazdırma biriktiricisi (spooler) bulunur; bir işletim sisteminde tek bir dosya sistemi ve pencere yöneticisi bulunur.
  - Çözüm 1: tek olacak nesne `static` tanımlanır. Sorun: nesne farklı sınıflarca kullanılacaksa hangisinin üyesi olacak? Birçok farklı türden nesne erişecekse Feature Envy kusuru ortaya çıkar.
  - Önerilen çözüm: sınıfa kendisinin tek örneğinin olmasını sağlama sorumluluğu atanır. Sınıfın kendi türünden `static` bir üyesi (`uniqueInstance`) olur, kurucu `private` tanımlanır, bu üye için `static` bir factory metodu (`getInstance()`) bulunur. `getInstance()` içinde nesne henüz yoksa oluşturulur, sonra döndürülür.
  - Singleton sınıflarından kalıtımla alt sınıflar türetilebilir; ilk akla gelen `static` üye çözümünde bu esneklik yoktur. Nesneyi tek bir türden nesne kullanacaksa `static` üye çözümü de sakıncasızdır.
  - İleride görülecek Strategy ve State kalıplarındaki somut alt sınıflar Singleton adayıdır, ama bu şart değildir.
- Adapter (ekrandan, 01:10:00 ve 01:12:30)
  - Amaç: işlev açısından uyumlu ama metot adları açısından uyumsuz bir istekçi sınıf ile bir sunucu sınıfı, birbirleriyle tek yönlü konuşturmak.
  - Benzetme: İngiliz standardındaki elektrik prizine Avrupa standardındaki cihazı takmak için araya bir adaptör konur.
  - Yapı: `Client` → `Target` (`Request()`); `Adapter`, `Target`'tan kalıtımla türetilir ve bir `adaptee` üyesi tutar; `Request()` yalnız `adaptee->SpecificRequest()` çağrısını iletir. `Adaptee` uyarlanması istenen sunucu sınıftır.
  - Bu yalın haldir: `Target`'ın ya da `Request()`'in soyut olması şart değildir. Parametre sırası farklıysa adaptörde sıra çevrilebilir.
  - Uygulanabileceği an: bir sınıfın istekçi tarafından kullanılmasının önündeki tek engel, mesaj adlarının (ve parametre sırasının) farklı olmasıdır.
- Adapter örneği: çizim programına hazır daire sınıfı (ekrandan, 01:14:50 ve 01:20:00)
  - Soyut `Shape` (`setLocation()`, `getLocation()`, soyut `display()`, `fill()`, `setColor()`, `undisplay()`) altında `Point`, `Line`, `Square` yazılmış. Daire yazılacakken elde kaynak kodu olmayan, yalnız derlenmiş hali olan hazır bir `XXCircle` sınıfı (`displayIt()`, `fillIt()`, `setItsColor()`, `undisplayIt()` ...) bulunuyor.
  - Çözüm: `Shape`'ten türeyen `Circle` sınıfı `private XXCircle myXXCircle` üyesi tutar, kurucuda onu oluşturur ve `display()` içinde `myXXCircle.displayIt()` çağırır.
  - Hoca, şemadaki ilişkinin elmas yerine sahiplik (aggregation) olarak çizilmesinin daha doğru olacağını, şemayı kaynaktan hazır aldığını söyledi.

- Bridge: amaç, sorun ve çözüm (ekrandan, 01:24:00 ve 01:30:00)
  - Amaç: nesnenin arayüzü ile gerçeklemesini birbirinden ayırmak; gerçekte ayırmak değil, istemciyi gerçeklemeden soyutlamak. Alternatif adı Handle'dır (Windows'taki handle mantığı bu kalıbın bir gerçeklemesidir).
  - Sorun: `Window` altında `XWindow` ve `PMWindow` varken bir `IconWindow` istenirse onun da `XIconWindow` ve `PMIconWindow` alt sınıfları gerekir. GNOME, KDE gibi yeni pencere yöneticileri ya da başka pencere çeşitleri eklendikçe sınıf sayısı katlanarak artar.
  - Çözüm: değişen iki şey (pencere soyutlaması ve gerçeklemesi) ayrı kalıtım hiyerarşilerinde sarmalanır. Soyut `Window` (`DrawText()`, `DrawRect()`) bir `imp` üyesiyle soyut `WindowImp`'e (`DevDrawText()`, `DevDrawLine()`) bağlıdır. `DrawRect()` dört kez `imp->DevDrawLine()` çağırır. `IconWindow` (`DrawBorder()`) ve `TransientWindow` (`DrawCloseBox()`) üst sınıf metotlarını kullanır; `XWindowImp` `XDrawLine()` ve `XDrawString()` ile, `PMWindowImp` PM'nin ilgili hizmetleriyle çalışır.
  - Çağrının soyutlamadan gerçeklemeye iletilmesi, araya bir köprü kurmak olarak görülür; kalıbın adı buradan gelir.
- Bridge'in uygulanabileceği durumlar (ekrandan, 01:32:40)
  - Bir soyutlama ile gerçeklemeleri arasındaki bağın kalıcı olmasının istenmediği (çalışma anında değiştirilebilmesi istenen) durumlarda.
  - Hem soyutlamaların hem gerçeklemelerin ayrı ayrı özelleştirilmesine gerek duyulduğunda. Hocaya göre kalıbın asıl çözdüğü sorun bu ikinci maddedir.
- Bridge örneği: iki çizim kitaplığıyla şekiller (ekrandan, 01:40:00, 01:42:40, 01:50:00, 01:50:30, 01:52:10)
  - İki dikdörtgen ailesi iki farklı kitaplıkla çizilir: `DP1.draw_a_line(x1, y1, x2, y2)` ve `DP2.drawline(x1, x2, y1, y2)`. Her şekil için ayrı Adapter kurmak mantıklı değildir.
  - Çözüm 1 (kalıtım): soyut `Rectangle` (`draw()` dört `drawLine` çağırır, `abstract protected drawLine(...)`), `V1Rectangle` DP1'i, `V2Rectangle` DP2'yi kullanır. Daire eklenince `Shape` altında `Rectangle`, `Circle` ve her biri için V1/V2 alt sınıfları oluşur.
  - Zayıf yön: yeni bir şekil ya da yeni bir kitaplık ailesi her seferinde birden çok sınıf gerektirir; şekil türleri ile çizim yolları sıkı birlikte tasarlanmıştır.
  - Bridge ile çözüm: `Shape Abstraction` (soyut `Shape`: `+draw()`, `#drawLine()`, `#drawCircle()`; altında `Rectangle`, `Circle`) ve `Drawing Implementation` (soyut `Drawing`: `drawLine()`, `drawCircle()`; altında `V1Drawing` → DP1, `V2Drawing` → DP2). `Shape`, `private Drawing _dp` köprü üyesini kurucuda alır.
- Örnekten çıkarılan dersler (01:53:06)
  - Neyin değiştiğini bul ve sarmala: şekiller ve çizimler ayrı soyutlamalara konur.
  - Kalıtım yerine parça-bütün ilişkisini tercih et: kalıtım çalışma anında değiştirilemez; sahiplik ilişkisi çalışma anında yeni bir nesne atanarak değiştirilebilir.

Hocanın Özellikle Vurguladığı Kısımlar

- Kalıp motamot uygulanmaz
  - Bir tasarım kalıbını kullanmak onu kelimesi kelimesine kullanmak demek değildir; kalıplar gereksinimlere göre değiştirilip özelleştirilerek tasarıma dönüştürülür.
- Protected kurucu ve protected setter'lar
  - Prototype'ta nesneler kurucuyla değil kopyalamayla oluşturulmalıdır. `private` kurucu alt sınıfları engellediği için `protected` kurucu tercih edilir. Kalıbın kendi tanımında kurucu konusu yer almaz, ama bunu düşünmek yararlıdır.
  - Setter'ların `protected` olması, statları değiştirmenin tek yolunun registry üzerinden kayıt olmasını sağlar (slayttaki not: "protected setter'lar ile erişimin tek yolu Register olur, tercih ederim").
- Üye alanları protected yapmak hocanın tercihi değil
  - Örnekte `Monster`'ın üye alanları `protected` tanımlanmıştı; hoca bunun güzel bir uygulama olmadığını, "dediğini de yap, yaptığını da yap" diyerek slaytta düzelteceğini söyledi.
- Derin mi sığ mı kopyalama gereksinime göre seçilir
  - Derin kopyalama her zaman daha iyi değildir: amir de ayrıca bir çalışan olarak kaydedilecekse derin kopyalama gereksiz iş ve yavaşlık getirir.
- Benzer kalıpların farkını bilmek
  - Bridge'in yaptığı iş belki Strategy gibi başka kalıplarla da yapılabilir; önemli olan kalıpların ayrıntılarını ve çözdükleri sorunları bilip uygun olanı seçmektir.

- Kalıtımın yanlış kullanımı büyük risk taşır
  - Özellikle gereksinimlerdeki değişiklik ve eklemelerde kalıtımın yanlış kullanımı sınıf sayısını patlatır.

Kısa Tekrar Notları

- Prototype: yeni nesneler bir prototipin `Clone()` ile kopyalanmasıyla oluşturulur. İstemci yalnız soyut `Prototype`'ı bilir.
- Prototype; sınıflar çalışma anında belirleniyorsa, paralel factory hiyerarşisi istenmiyorsa ya da durum uzayı sınırlıysa seçilir (biri yeterli).
- Reflection ya da late binding olan dillerde kalıp yerine dil yetenekleri kullanılabilir.
- Sığ kopyalama yalnız nesneyi, derin kopyalama ilişkili nesneleri de kopyalar; derin kopyalamada çevrimselliğe dikkat.
- Level editörü örneği: `Monster` (Prototype), `GenericMonster` (ConcretePrototype), `MonsterRegistry` (`HashMap` ile kayıt), `MainApp` (Client).
- Singleton: `static uniqueInstance`, `private` kurucu, `static getInstance()`. Tek örnek + ortak erişim noktası.
- Adapter: `Adapter`, `Target`'tan türer ve `Request()` çağrısını `adaptee.SpecificRequest()`'e iletir. Tek engel metot adı ya da parametre sırası farkıysa kullanılır.
- Adapter örneği: `Circle extends Shape` içinde `XXCircle` üyesi; `display()` → `displayIt()`.
- Bridge: soyutlama ve gerçekleme ayrı hiyerarşilerde, aralarında köprü üyesi (`imp`, `_dp`). Diğer adı Handle.
- Bridge örneği: `Shape/Rectangle/Circle` ile `Drawing/V1Drawing/V2Drawing`; DP1 ve DP2 kitaplıklarının farklı parametre sırası `V1Drawing/V2Drawing` içinde ele alınır.
- Kalıtım çalışma anında değişmez; parça-bütün ilişkisi değişebilir.

Detaylı Açıklamalar

- Prototype'ın müzik besteleme örneğinde asıl sorun, çerçeve programının (framework) yazıldığı sırada nota sınıflarının henüz var olmamasıdır. `GraphicTool` hangi sınıftan nesne oluşturacağını bilemez, ama elinde bir prototip nesne varsa ona "kendini kopyala" diyebilir. Kopyalama işi, her somut sınıfın kendi `Clone()` gerçeklemesine bırakılır. Böylece her nota türü için ayrı bir araç sınıfı ya da ayrı bir fabrika yazmak gerekmez; aynı `GraphicTool` sınıfının farklı prototiplerle yapılandırılmış nesneleri yeterlidir.
- Level editörü örneğinin kodu (ekrandan, 00:35:10 ve 00:38:30):

```
package dp.prototype.example;
public abstract class Monster {
    protected int HP, DEX, AGL, LUCK, CHR;
    protected String type;

    public abstract Monster clone( );

    protected Monster(int hp, int dex, int agl, int luck, int chr) {
        HP = hp; DEX = dex; AGL = agl; LUCK = luck; CHR = chr;
    }
    public String getType() { return type; }
    public int getHP() { return HP; }
    protected void setHP(int hP) { HP = hP; }
    // getDEX/setDEX, getAGL/setAGL, getLUCK/setLUCK, getCHR/setCHR aynı biçimde
    protected void setType(String type) { this.type = type; }
}

public class GenericMonster extends Monster {
    public GenericMonster(int hp, int dex, int agl, int luck, int chr) {
        super(hp, dex, agl, luck, chr);
        type = "Generic Monster";
    }
    public Monster clone( ) {
        return new GenericMonster(HP, DEX, AGL, LUCK, CHR);
```

```
    }
}

public class MonsterRegistry {
    private HashMap<String, Monster> monsters;
    public MonsterRegistry() { monsters = new HashMap<String,Monster>(); }

    public Monster findMonster( String type ) {
        return monsters.get( type );
    }
    public void registerMonster( Monster m ) {
        monsters.put( m.getType(), m );
    }
}
```

- `MainApp` istemci rolündedir (ekrandan, 00:40:00): registry oluşturulur, `new GenericMonster(10, 10, 10, 10, 10)` kaydedilir, `reg.findMonster("Generic Monster").clone()` ile kopya alınır, kopyanın `setHP(100)` ve `setType("Bolum sonu canavarı")` ile değerleri değiştirilir ve `reg.registerMonster(mySpecialMonster)` ile yeniden kaydedilir. Bundan sonra "Bolum sonu canavarı" anahtarıyla bu canavar istenebilir.
- Hoca, istemcinin `GenericMonster` kurucusuna doğrudan eriştiğine dikkat çekti. Kalıpta istemci yalnız `Prototype` ile muhataptır. Bir öneri olarak somut prototipin kurucusu `private` yapılabilir ve registry ilk oluşturulduğunda varsayılan bir canavarı kendisi kaydedebilir. Ama erişimi kısıtlamaya gerek yoksa kurucunun public kalması da savunulabilir; karar, problemin gereksinimlerine bağlıdır.
- Hoca, setter'ların protected yapılmasını açıklarken Java'da `protected`'ın alt sınıflara ve aynı paketteki diğer sınıflara public gibi göründüğünü hatırlattı.

  Ders sonrası not: Hoca bir yerde (00:31:32) protected için "alt sınıflara public, diğer sınıflara private gibi davranır" dedi, sonra (00:35:44) aynı paketteki sınıfları da ekledi. Java'da doğrusu ikincisidir: `protected` üyeye alt sınıflar ve aynı paketteki bütün sınıflar erişebilir.

- Singleton'da `getInstance()` notu (ekrandan, 00:50:00): `if( uniqueInstance == null ) { uniqueInstance = new Singleton( ... ) } return uniqueInstance;`. `uniqueInstance` static olduğu için `getInstance()` da static olmak zorundadır. Hoca kalıbın basit olduğunu söyleyip ayrı bir kod örneği yapmadı.
- Adapter'da `Adapter`, `Target`'tan türediği için istemci onu `Target` gibi kullanır; adaptör içindeki tek iş, gelen çağrıyı uyarlanan nesnenin karşılık gelen metoduna iletmektir. Daire örneğinde bu, kaynak kodu olmayan bir sınıfı yeni çerçeveye uydurmayı sağlar: `display()` adının `displayIt()` olması artık sorun değildir (ekrandan, 01:20:00).
- Bridge örneğinde çözüm 1'in dikdörtgen kodu (ekrandan, 01:40:00):

```
package bridge.solution1;
public abstract class Rectangle {
    private double _x1, _x2, _y1, _y2;
    public void draw( ) {
        drawLine(_x1,_y1,_x2,_y1);
        drawLine(_x2,_y1,_x2,_y2);
        drawLine(_x2,_y2,_x1,_y2);
        drawLine(_x1,_y2,_x1,_y1);
    }
    abstract protected void drawLine ( double x1, double y1,
        double x2, double y2);
}
public class V1Rectangle extends Rectangle {
    protected void drawLine(double x1, double y1,
                double x2, double y2) {
        DP1.draw_a_line(x1,y1,x2,y2);
    }
}
```

- `drawLine` public değil protected yapılmıştır, çünkü dikdörtgenin çizgileri hangi kitaplıkla çizdiği nesnenin iç işleyişine aittir; protected ayrıca alt sınıflara aktarım avantajı verir.
- Bridge ile çözümde (ekrandan, 01:50:30 ve 01:52:10) `Shape` köprü üyesini tutar ve çizim isteklerini ona iletir; somut şekiller yalnız hangi temel çizim metodunun hangi sırayla çağrılacağını bilir:

```
package bridge.solution2;
public abstract class Shape {
    public abstract void draw( );
    private Drawing _dp;
    public Shape (Drawing dp) { _dp= dp; }
    protected void drawLine(double x1, double y1,
      double x2, double y2) { _dp.drawLine(x1, y1, x2, y2); }
    protected void drawCircle(double x,double y,
      double r ) {_dp.drawCircle(x, y, r); }
}
public class Circle extends Shape {
    private double _x, _y, _r;
    public Circle (Drawing dp,
              double x,double y,double r) {
        super( dp) ;
        _x= x; _y= y; _r= r ;
    }
    public void draw () { drawCircle(_x,_y,_r); }
}

public abstract class Drawing {
    public abstract void drawLine( double x1, double y1,
              double x2, double y2 );
    public abstract void drawCircle (double x,double y,
              double r);
}
public class V1Drawing extends Drawing {
    public void drawCircle(double x, double y, double r) {
        DP1.draw_a_circle(x,y,r);
    }
    public void drawLine(double x1, double y1,
              double x2, double y2) {
        DP1.draw_a_line(x1,y1,x2,y2);
    }
}
```

- `V2Drawing` de aynı biçimde DP2'nin `drawline` ve `drawcircle` metotlarını, DP2'nin beklediği parametre sırasıyla çağırır. Böylece V1Circle, V2Circle, V1Rectangle, V2Rectangle gibi sınıflara gerek kalmaz.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.